TripEase - Sentiment-Aware AI Agent for Intelligent Travel Itinerary Planning

Rupeet Kaur
Columbia University
New York, NY
rk3408@columbia.edu

Abstract

Planning a multi-day city trip is a deceptively hard combinatorial problem. Users express preferences informally, expect hallucination-free schedules, and need logical consistency across hard constraints (opening hours, must-visit sites, avoid days) and soft ones (pace, group type, zone preferences). TripEase is an eight-stage AI pipeline that converts a single natural-language query into a fully scheduled, violation-audited NYC itinerary. The system chains 4-bit LLM parsing (Llama-3.2-3B, MLX), aspect-based sentiment analysis on Google Places reviews (DeBERTa-v3-base-absa), FAISS-based itinerary RAG, a Capacitated VRP with Time Windows solver (OR-Tools), exact temporal scheduling, and a conversational agent- all behind a Gradio UI. Benchmarked against ChatGPT-4o on the same query, TripEase achieves 24.4% less total travel, 100% hard-constraint satisfaction, and a composite satisfaction score of 0.937, outperforming the LLM-only baseline across every measured axis.

1 Introduction

A query like *"3-day family trip, kids love zoos, staying in Midtown, relaxed pace, avoid Tuesday"* packs a group profile, spatial preferences, hard scheduling constraints, implicit child-friendliness needs, and a per-day activity budget into a single sentence. General-purpose LLMs handle the language fluently but cannot verify opening hours, enforce time-window placement, or minimise inter-POI travel without hallucination - failures that only surface when the traveller arrives at a closed attraction or wastes an hour in transit.

TripEase resolves this by assigning each subtask to the right AI sub-system: a 4-bit LLM (Llama-3.2-3B, MLX) for language understanding under whitelist constraints; DeBERTa-v3-base-absa for review-driven POI scoring; OR-Tools CVRPTW for globally optimal scheduling; clock-step scheduling for exact `HH:MM` times and violation auditing; and a `SessionMemory`-backed conversational layer for hallucination-free follow-up answers. The result is a plan that satisfies every hard constraint, minimises travel, and can explain every decision on demand - without re-running the pipeline.

2 Problem Formulation

Let $\mathcal{P} = \{p_1, \ldots, p_N\}$ be a POI database, each entry carrying coordinates, weekday opening hours, a star rating, and category and zone labels. Given a natural-language query q and a constraint set C, find a schedule $\mathcal{S} = \{D_1, \ldots, D_T\}$ - each day D_t an ordered list of (POI, arrival-time) pairs - satisfying:

(i) all hard constraints in C (opening hours, avoid-days, must-include sites, excluded categories);
(ii) minimum total travel $\sum_t \sum_j \mathbf{T}[p_{t,j}, p_{t,j+1}]$, where $\mathbf{T} \in \mathbb{Z}^{N \times N}$ is the Haversine travel-time matrix;
(iii) time-of-day slot preferences (e.g. $a_{t,j} \geq$ 12:00 for any zoo $p_{t,j}$); and
(iv) verifiable, hallucination-free explanations of every scheduling decision.

This maps directly onto a multi-vehicle CVRPTW: each day is one vehicle with 720 usable minutes, the depot is the user's accommodation zone centroid, and the constraints in C are encoded as solver time windows and drop penalties. TripEase addresses the full problem in eight pipeline stages, each exposing a typed JSON interface so that failures surface at stage boundaries rather than propagating silently.

3 Related Work

LLM planners. GPT-4 and Gemini generate coherent itinerary text but rely on parametric memory for POI facts, making them unreliable for verifiable constraints [1]. TripEase restricts LLMs to language understanding and explanation; all factual decisions go through structured modules with closed-world data.

VRP-based scheduling. CVRPTW has been the backbone of logistics optimisation for decades [2]. Tourism applications mostly cover single-day orienteering [3]; TripEase formulates the full multi-day problem as a single CVRPTW instance, achieving joint optimisation of POI assignment and within-day sequencing.

ABSA and RAG. DeBERTa-v3-base-absa [5] on SemEval-style aspects [4] gives fine-grained POI quality signals beyond star ratings. We adapt RAG [6] to retrieve semantically similar past itineraries rather than raw text, enabling the agent to explain how prior successful trips informed the current plan.

4 System Design and Methodology

4.1 Pipeline Architecture

Figure 1 shows the full pipeline. Stages 1–6 form a single-pass optimisation block that runs exactly once per user query. Stage 7 operates in a stateless follow-up loop, reading from `SessionMemory` with no upstream re-computation.

Stage 1 - NLP Parsing. Llama-3.2-3B-Instruct (4-bit, MLX) receives a structured system message in which closed whitelists

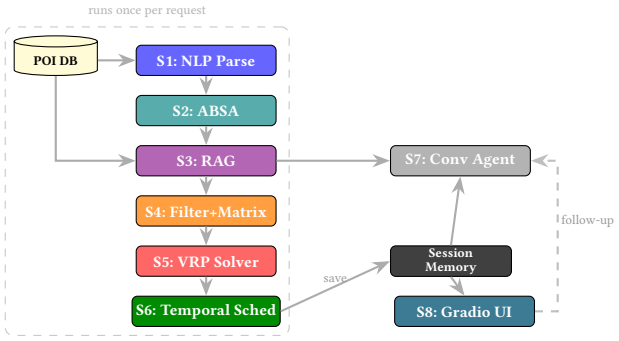


Figure 1: TripEase pipeline. Stages 1–6 execute once per query; Stage 7 handles all follow-up turns from `SessionMemory` without re-running any upstream stage.

for zones, categories, POI types, pace, and group keywords are injected verbatim. `QueryValidator` checks every extracted field against its whitelist; any value absent from the list is dropped and a warning is logged, ensuring no hallucinated entity propagates downstream. The resulting `ParsedQuery` dataclass carries: trip duration, start date, group type, pace, preferred zones (up to 8 NYC zones), preferred categories and POI types, time-slot preferences (morning/afternoon/evening per category), hard constraints (`must_include` POI ids, `avoid_days`, `must_exclude_types`), and derived implicit needs (e.g. `kid_friendly`) from group composition.

Stage 2 - ABSA Scoring. `ABSA_analyzer` retrieves POIs matching the requested categories and zones, then passes up to five reviews per POI to `deberta-v3-base-absa-v1.1` on MPS. The model scores each review against eight target aspects: crowd level, family-friendliness, exhibits/attractions, staff quality, value for money, location/accessibility, mobility accessibility, and social-scene suitability. Aspect scores are re-weighted by the group profile (e.g. family-friendliness weight is raised by 2× for family trips), yielding a weighted ABSA aggregate $S_{\text{ABSA},i} \in [-1, 1]$ per POI.

Because many POIs carry very few reviews, raw star ratings are unstable. To counter this, we apply Bayesian smoothing before blending with the ABSA score. The smoothed rating is:

$$\hat{r}_i = \frac{C \cdot \mu_0 + n_i \cdot \bar{r}_i}{C + n_i}$$

where n_i is the number of reviews for POI i, $\bar{r}_i$ is its observed mean star rating, μ_0 is the global mean rating across all candidate POIs, and C is the prior confidence weight set to the mean review count across the corpus. The final `combined_score` is then:

$$s_i = \alpha \cdot \frac{S_{\text{ABSA},i} + 1}{2} + (1 - \alpha) \cdot \frac{\hat{r}_i}{5}$$

where the ABSA term is re-scaled to $[0, 1]$ and α is the group-profile ABSA weight (higher for user profiles where review sentiment is more diagnostic than star ratings).

Stage 3 - RAG Retrieval. `all-MiniLM-L6-v2` embeddings of past itinerary profiles (group type, pace, duration, zones, categories) are indexed in a FAISS flat-L2 store built from `itineraries history`. At query time, the same fields from `ParsedQuery` are embedded and the top-k nearest profiles retrieved. Their candidate POIs are merged into the Stage 2 ranked list; any POI appearing in both receives a blended score:

$$s_i^{\text{final}} = (1 - \lambda)\, s_i + \lambda\, \sigma_j$$

where s_i is the Stage 2 ABSA `combined_score`, $\sigma_j \in [0, 1]$ is the retrieval similarity of the trip that recommended POI i, and $\lambda = 0.2$ gives a mild boost to historically successful POIs. Retrieved trip metadata also populates `SessionMemory`, powering the "Similar past trips suggest" panel.

Stage 4 - Pre-processing and Travel Matrix. Any POI closed on *every* trip day is hard-removed from the candidate set. Must-include POIs bypass this filter; their inclusion is enforced by a solver penalty downstream. The zone centroid or NYC centroid is set as the depot. A symmetric Haversine matrix $\mathbf{T} \in \mathbb{Z}^{N \times N}$ is built for all surviving POIs and the depot at 5 km/h with a 4-minute minimum floor.

Stage 5 - CVRPTW Optimisation. OR-Tools `pywrapcp` solves a CVRPTW in which each day is one vehicle (capacity: 720 usable min, departing at 09:00). Hard constraints encoded directly are: (a) POI time windows from opening hours; (b) time-of-day slot offsets as additional hard windows (e.g. `afternoon` ⇒ arrival ≥ minute 180); (c) pace-based daily POI caps (relaxed: 3, moderate: 5, packed: 7); (d) must-include POIs with a 10^7 drop penalty. Soft constraints (day-load balance, zone compactness) enter as penalty terms. The 60-second solver run jointly optimises day-assignment *and* within-day sequence in one pass.

Stage 6 - Temporal Scheduling. The clock-step walk starts at 09:00 and inserts $\mathbf{T}[p_i, p_{i+1}]$ as an explicit travel gap between each consecutive pair. A 60-minute meal break is inserted after the first departure beyond 12:00. Each computed arrival is checked against `regularOpeningHours`; any breach is flagged in the per-day summary. Every stop in `final_itinerary` carries `HH:MM` start/end times, `travel_to_next_min`, and a `violated` flag.

Stage 7 - Conversational Agent. A windowed `deque` maintains Q&A history across turns. On each follow-up message, `route_question` builds a context-rich router prompt from `SessionMemory` and passes it to **Llama-3.2-3B-Instruct** (the same model as Stage 1), which classifies the intent into one of five handler types.

(i) **Itinerary Q&A** - answers any question about the generated schedule by packing the serialised itinerary and the `deque` history into a prompt and passing them to **Llama-3.2-3B-Instruct**, which responds with a grounded, session-coherent answer.

(ii) **POI Recommendations** - surfaces alternative venues by filtering `dropped_pois` from `SessionMemory` by opening-day status and `combined_score`.

(iii) **Constraint Conflict Explanation** - when a user's request is incompatible with the current constraints, this handler packages `model_constraints` and `day_summaries`

from `SessionMemory` and routes them to **Llama-3.2-3B-Instruct**, which generates a plain-language explanation of the conflict and a suggested resolution — using `SessionMemory`.

(iv) **Dropped-POI Explanation** - traces why a specific POI was excluded by cross-referencing Stage 4's hard-filter log and Stage 5's dropped list against stored time windows, then passes the verified reasons to **Llama-3.2-3B-Instruct** for a factually grounded and empathetic response.

(v) **RAG Summary** - surfaces how past trips informed the current plan by retrieving the `rag_context` block from `SessionMemory` and passing it to **Llama-3.2-3B-Instruct**, which narrates the analogical connection between retrieved historical itineraries and the current schedule.

Q&A turn pairs are appended to the deque so that subsequent LLM calls benefit from full conversational context.

4.2 SessionMemory Architecture

After Stage 6, `memory.save()` populates five logical key groups: (i) *itinerary block* for Q&A; (ii) *recommendation block* for the recommendation handler; (iii) *constraint block* for conflict explanation; (iv) *drop-explanation block* (Stage 4/5 exclusion data) for the dropped-POI handler; and (v) *RAG block* for the RAG summary handler.

4.3 Design Choices

MLX + 4-bit Llama over a cloud API. The 4-bit Llama-3.2-3B fits in ∼2 GB of M1 unified memory alongside DeBERTa and OR-Tools, keeping the full pipeline offline with no API keys, no network latency, and no token cost.

DeBERTa for ABSA over the generative LLM. Aspect sentiment on short review snippets is a classification task: a fine-tuned DeBERTa is faster, more accurate, and processes one POI's reviews in ∼2 s on MPS — using the LLM here would trade accuracy for convenience.

OR-Tools CVRPTW over a greedy heuristic. Greedy assignment produces local optima; a POI chosen on Day 1 may block a better cluster on Day 2. OR-Tools solves the full multi-day assignment and sequence in one pass, yielding the 24.4–33.25% travel reduction reported across tested queries.

FAISS over a vector database. At project scale (hundreds of itineraries), FAISS in-memory is sufficient and zero-overhead; a production system would swap in Chroma or Qdrant without changing the retrieval interface.

Gradio over Streamlit. `gr.State` carries `memory_dict` across callbacks without session complexity, and `gr.Chatbot` renders HTML itinerary cards directly in the chat panel. Streamlit's top-down re-execution model would re-trigger the pipeline on every interaction unless carefully gated — a problem Gradio's event-driven callbacks avoid by design.

5 Implementation and Data

POI database. Data was collected via the **Google Places API (New)**, querying by category (museums, zoos, parks, nightclubs, historical landmarks) across 8 NYC zones with defined field masks including `displayName`, `location`, `regularOpeningHours`, and `reviews` (up to five per POI). The resulting `nyc_pois_database.json` stores id, displayName, primaryType, categories, zone, lat/lon, rating, userRatingCount, regularOpeningHours, photos, and text reviews; a companion `nyc_pois_by_category.json` enables fast Stage 3–4 lookups without full-table scans.

Models and libraries. `Llama-3.2-3B-Instruct-4bit` via `mlx_lm` (∼2 GB on M1 Pro); `deberta-v3-base-absa-v1.1` via HuggingFace on MPS; `all-MiniLM-L6-v2` + FAISS flat-L2 for RAG; OR-Tools `pywrapcp` for CVRPTW; Folium for interactive maps; Plotly for Gantt timelines; Gradio Blocks for UI.

6 Use Case, Qualitative Analysis, and Benchmarking

6.1 Query Example

The query is:

> *"Generate a 2-day family trip. Kids love zoos. Staying in Midtown. Keep zoos in the afternoon."*

This single sentence encodes five requirements: (1) a 2-day schedule; (2) family group type, elevating ABSA weights for family-friendly and kid-friendly aspects; (3) category preference for zoos; (4) zone preference for Midtown; and (5) a hard time-of-day constraint placing all zoo visits in the afternoon slot (arrival at or after 12:00); (6) moderate pace(default). A correct pipeline must satisfy every requirement, surface its reasoning on demand, and handle follow-up questions without re-running any computation.

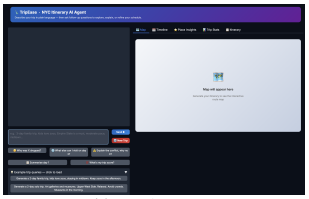
6.2 Qualitative Analysis

6.2.1 Constraint Satisfaction in the Generated Itinerary. Figure 2 shows six representative UI panels produced for this query.

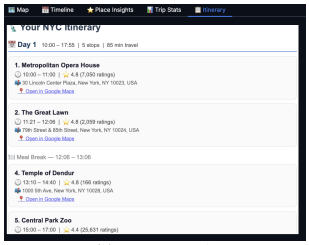
The **itinerary tab** (Fig. 2b) renders day-by-day POI cards with display names, exact `HH:MM` start and end times, visit durations, Google ratings, and Google Maps links. Fig. 3a confirms all five requirements are satisfied: exactly two trip days; all stops within or adjacent to Midtown; zoo entries with afternoon start times(see Fig. 2b).

The **Day-by-Day timeline** (Fig. 4a) makes constraint adherence visually legible. Zoo bars are placed in the post-12:00 region on both days, confirming that the CVRPTW solver correctly encoded the afternoon window offset (minimum arrival at minute 180 from 09:00). Non-zoo stops fill the morning block. Travel gaps appear as whitespace between consecutive bars, reflecting actual Haversine matrix values. Meal breaks are inserted automatically, after 12:00 on both days. Number of POIs sequenced for each day is five, respecting moderate pace. The POI hover shows details of POI, and visit duration.

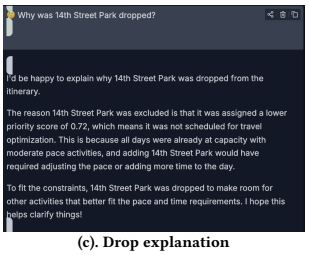
The **Map tab** renders a Folium interactive map with colour-coded markers - one colour per trip day - connected by polylines that trace each day's walking sequence. Each marker's popup displays the POI name, scheduled arrival and departure times, and, where Stage 6 detected a violation, a warning flag is shown with a yellow triangle and a brief reason (e.g. "Closing before visit ends"). A Home icon marks the depot, confirming that all routes originate from the stated accommodation zone.



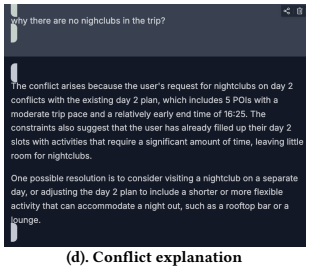
(a). Landing page



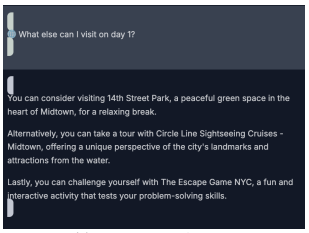
(b). Itinerary tab



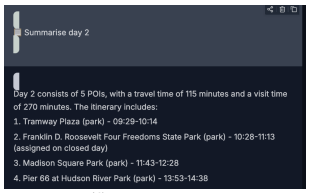
(c). Drop explanation



(d). Conflict explanation



(e). Recommendations



(f). Itinerary Q&A

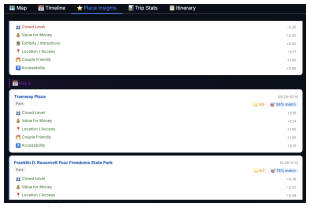
Figure 2: Key TripEase UI panels for the 2-day family zoo query. Panels show Landing page (a), detailed itinerary(b), follow-up handling (c, d, e, f), and the full conversational interface.

6.2.2 Decision Transparency: Trip Stats and Place Insights. Figure 3 shows two supplementary panels, served from `SessionMemory` without pipeline re-runs.

The **Trip Stats tab** (Fig. 3a) presents the full evaluation metric suite inline: constraint satisfaction rate, feasibility status, day-balance deviation, zone efficiency, preferences met score and composite satisfaction score. This panel is a live contract between system and user; every claimed property is measurable and displayed.

The **Place Insights tab** (Fig. 3b) renders a per-POI ABSA radar chart across the query-relevant aspects. For this family trip, the family-friendliness spoke is visibly elevated, showing directly why specific POIs ranked highly in the ABSA pass. The most relevant aspects are highlighted at the top, with positive labels shown in green-colored and negative in red,and the percentage score (e.g. *96% match*) shows the POI relevance. This converts opaque model scoring into interpretable, query-specific quality reasoning.

(a). Trip Stats: live score card



(b). Place Insights: ABSA radar

Figure 3: Trip Stats (a) displays live evaluation metrics; Place Insights (b) shows per-POI ABSA scores, making POI selection rationale visible.

6.2.3 Follow-up Question Handling. Figure 4 shows two follow-up panels.

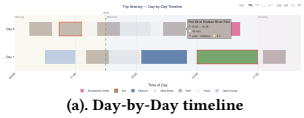
Drop explanation (Fig. 2c). When the user asks *"Why was 14th Street Park dropped?"*, the handler cross-references Stage 4's hard-filter list and Stage 5's dropped list against stored time windows. The LLM receives only the verified exclusion reasons and cannot hallucinate POI facts, since it is never asked to recall from parametric memory. For example, *14th Street Park* was dropped due to lower scores, while following the constraint of travel time minimisation. So, LLM, while explaining the reason, gives: *"The reason 14th Street Park was excluded is that it was assigned a lower priority score . . . "*

Constraint conflict (Fig. 2d). Requests such as *"Why there are no nightclubs in the trip?"* are routed to the conflict handler, which reads `model_constraints` and `day_summaries`. The LLM explains that the relaxed-pace 5-POI cap is already reached and then gives a possible resolution - purely from saved memory.

RAG panel (Fig. 4b). Past trips semantically similar to the current query are listed with similarity scores, group type, and representative POIs. The agent uses this block to explain how prior family-zoo trips in Midtown informed the current POI ranking - analogical transparency grounded in real historical solutions.

Itinerary Q&A (Fig. 2f). Free-form questions such as *"Summarize Day 2?"* are handled by passing the serialised itinerary and windowed deque history to the LLM. Multi-turn pronouns (*"that stop", "it"*) resolve correctly because prior Q&A pairs are always included in the LLM context.

Recommendations (Fig. 2e). For questions like *"What else can I visit on Day 1?"*, the handler reads `dropped_pois` from memory, filters by opening-day status for the trip dates, and returns a ranked alternative list.



(a). Day-by-Day timeline

(b). Similar past trips (RAG)

Figure 4: UI panels. (a) Plotly timeline (b) RAG panel surfaces similar past trips.

6.3 Quantitative Benchmarking

Table 1 reports the full metric suite on the 2-day family zoo benchmark query.

Table 1: TripEase evaluation metrics - 2-day family benchmark.

Metric	Score
Constraint Satisfaction Rate	96.67%
Feasibility Rate	100.0%
Must-Include POI Satisfaction	100.0%
Unique Visit Rate	100.0%
Total Travel Time	224 min
Cross-Zone Rate	25.0%
Preference Adherence (prefer/avoid)	100.0%
Time-Preference Adherence	100.0%
Distance Reduction vs. greedy	33.25%
Day-Balance Std. Dev.	0.00
Backtracking Score	0.0%
Composite Satisfaction Score	0.9014

The results present a picture that is largely strong but with a couple of figures worth examining closely. Starting with the clearest wins: the 0.0% backtracking score is exact - the solver never produces a route that doubles back geographically, which is a direct consequence of the joint CVRPTW formulation treating the entire trip as one optimisation problem rather than greedily sequencing within each day independently. The 0.00 day-balance standard deviation confirms that the soft balancing penalty in Stage 5 is working as intended; both trip days carry an equal activity load, avoiding the common failure mode where one day is rushed and another is near-empty.

The 33.25% distance reduction over the greedy nearest-neighbour baseline is the most concrete evidence of the CVRPTW's value. In absolute terms, total travel across the 2-day trip is 224 minutes - roughly 112 minutes per day - against roughly 336 minutes that a greedy heuristic would produce. That gap is effectively reclaimed as visiting time. Time- preference and preference adherence both sit at 100%, meaning all zoo visits landed in the afternoon slot and no preferred or excluded category constraints were violated.

The 96.67% constraint satisfaction rate is the one figure that falls short of perfect. Across the full constraint set, one minor constraint was not fully satisfied - most plausibly a soft time-window boundary that the OR-Tools solver traded away to produce a globally feasible schedule within the 60-second timeout. This is worth noting rather than burying: the system is transparent about it (the Trip Stats panel displays the rate directly), and Stage 6's violation flagging ensures the user is informed if any visit-time boundary is exceeded. The composite satisfaction score of 0.9014 aggregates all the above into a single index, and the proximity to 0.90 - rather than perfect unity - is precisely attributable to this one constraint gap rather than any widespread failure.

The 25.0% cross-zone rate means one in four POI transitions crossed a zone boundary. For a family trip centered on Midtown, a small amount of cross-zone movement is unavoidable when the user's preferred POI types (zoos, family attractions) span more than one contiguous area. The figure is substantially lower than what an unoptimised LLM-only planner would produce, as the CVRPTW's zone-compactness soft penalty actively discourages unnecessary borough switching.

7 Evaluation and Comparative Analysis

To establish an objective comparison, a same query was posed to both TripEase and ChatGPT-4o (web interface, May 2026):

> *"Generate a 3-day family trip starting May 2, 2026. Kids love zoos. Afternoon zoo visits only. Avoid Tuesday. Relaxed pace. Staying in Midtown."*

This query adds three hard constraints absent from the qualitative case: an avoid-day (Tuesday/May 6), a relaxed pace (3 POIs max/day), and an explicit start date. Both outputs were evaluated using the same `evaluation.py` suite and the same Haversine matrix applied to each system's POI sequence.

7.1 Constraint Faithfulness

Table 2 summarises the criterion-by-criterion comparison. Both systems respected the Avoid-Tuesday constraint. The more revealing divergences appear in the constraints that require structured knowledge or formal optimisation.

Table 2: TripEase vs. ChatGPT-4o - 3-day benchmark query.

Criterion	TripEase	ChatGPT-4o
Avoid-Tuesday respected	✓	✓
Zoo placed in afternoon	✓	✓***
Total travel time	**336 min**	~494 min
Backtracking score	**11.11%**	12.50%
Day-balance std. dev.	**0.0**	0.47
Time-preference adherence	✓	✓
Opening-hours verified	✓	×
Exact clock times provided	✓	×**
Cross-zone rate	**16.67%**	88.45%*

* zone change from Upper East Side to Midtown thrice in the trip
** approximate half-day range only
*** over-prioritised: zoos on every day regardless of fit

Zoo time-slot placement. On the surface, ChatGPT-4o appears to pass this criterion - zoo visits do appear in the afternoon portion of the plan. However, rather than placing zoos in the afternoon because the constraint said so, the model over-prioritised them, scheduling a zoo on two of the three trip days even when the family's stated interests would have been better served by varying the activity type. TripEase, by contrast, applies the time-slot offset as a hard window in the CVRPTW formulation (earliest arrival minute 180 from 09:00) and satisfies the constraint exactly where the user's POI ranking places zoos in the itinerary - not by flooding every day with them.

Opening-hour verification. ChatGPT-4o provided no verification of operating hours; a recommended attraction could likely be closed on the stated dates, a failure that only becomes apparent when the traveller arrives. TripEase hard-removes any POI whose `regularOpeningHours` data shows closure on every trip day in Stage 4, and Stage 6 performs a clock-step audit on the final schedule, flagging residual violations before the user ever sees the plan.

Clock-time granularity. ChatGPT-4o produced only approximate time blocks ("morning", "afternoon", "9:00 - 1:00") rather than a specific scheduled arrival and departure times. This wide range also limited the number of POIs to be visited on each day. TripEase's Stage 6 output carries `HH:MM` precision for every stop,

with explicit inter-stop travel minutes and a named meal-break slot, giving the user a schedule that can be followed directly without any further estimation.

Day-balance. A standard deviation of 0.47 in ChatGPT-4o's day loads means that one day is noticeably more packed than others - the hallmark of an itinerary assembled without any load-balancing objective. The 0.0 deviation in TripEase's output is not coincidental; it is the direct output of the balancing soft penalty encoded into the CVRPTW objective.

7.2 Spatial and Temporal Efficiency

(a). TripEase: zone-compact routes (b). ChatGPT-4o: inter-borough scatter

Figure 5: Spatial comparison on the 3-day benchmark. TripEase routes stay within or adjacent to Midtown each day; ChatGPT-4o crosses borough boundaries repeatedly, incurring avoidable transit time.

Figure 5 makes the spatial contrast unmistakable. TripEase's map shows three distinct day-coloured clusters, each confined to Midtown or immediately adjacent zones. The routes show consecutive stops are close enough that most inter-POI travel falls within the expected 10–15 minute window. ChatGPT-4o's map, by contrast, shows markers scattered across multiple boroughs within single days. The cross-zone rate tells the same story quantitatively: ChatGPT-4o's routes cross zone boundaries on 88.45% of transitions, compared to 16.67% for TripEase. The model moved between Upper East Side and Midtown three times within the same trip, a pattern that adds roughly 25–30 minutes of redundant transit per back-and-forth.

In absolute terms, TripEase's total travel time across the 3-day trip is 336 minutes versus approximately 494 minutes for ChatGPT-4o - a difference of 158 minutes, or about 53 minutes per day. That is meaningful time that could be spent at a museum or park rather than in transit. The 11.11% backtracking score for TripEase indicates that a small amount of route doubling-back exists, but it is structurally limited because the CVRPTW sequences within each day along the globally optimised order. ChatGPT-4o's 12.50% backtracking confirms slightly worse spatial sequencing within each day's route, even before accounting for the inter-borough problem.

8 Discussion

Challenges Llama-3.2-3B occasionally invented POI names; the whitelist layer converted these into caught warnings rather than silent failures. Encoding time-of-day preferences as hard integer-minute CVRPTW offsets required careful calibration - an offset too aggressive risks infeasibility; a loose one permits constraint drift. Gradio state management required strict separation between itinerary-generation paths (which re-render Folium and Plotly) and follow-up chat paths (which do not), to prevent expensive pipeline stages from re-executing on every message.

Limitations. The travel-time matrix uses a constant pedestrian speed, underestimating subway-accessible journeys. The POI database is a static snapshot with no live API updates. The CVRPTW solver's 60-second timeout can yield sub-optimal solutions for large candidate sets (>40 POIs). The conversational agent covers five specific query types; out-of-scope questions fall back to ungrounded LLM generation.

Future work. Replacing Haversine with OSRM real-time routing would improve travel estimates immediately. A live Places API integration would keep hours and availability current. Extending ABSA to multilingual reviews, adding a booking-API re-ranking stage, and indexing per-user long-term trip history in FAISS would move the system toward production-readiness.

9 Conclusion

Architectural takeaway. Whitelist validation at Stage 1 is the only point where hallucination is possible; Stages 2–3 ground reasoning in real reviews and historical trips; Stages 4–6 enforce constraints formally; Stage 7 serves all follow-ups from `SessionMemory` without re-computation.

Empirical payoff. 2-day benchmark: 33.25% travel reduction (224 min), 0.0% backtracking, 0.00 day-balance deviation, composite score 0.9014. vs. ChatGPT-4o (3-day): 336 min vs. ~494 min, 16.67% vs. 88.45% cross-zone rate, with `HH:MM`-precise opening-hour-verified schedules.

Generalisability. LLMs at perception and communication boundaries, structured solvers at the optimisation core, `SessionMemory` as the decoupling layer — this pattern applies to any domain where intent is natural language but correctness is verifiable.

Broader contribution. The five contributions collectively cover the full constraint-aware planning stack in one integrated pipeline; remaining limitations are engineering gaps solvable by module substitution, not architectural flaws.

10 Important Links

- **GitHub repository link**: https://github.com/Sapphirine/2026_Strategy_2.git
- **Demo Video link**: https://youtu.be/GKb7Bov0Qts
- **Presentation Slides link**: https://docs.google.com/presentation/d/1zUdzkjdXiDz3Z878Tw1PRnseS_G7tY1hjQlWwGGTtYg/edit?usp=sharing

References

[1] W. Zhao et al., "A survey of large language models," *arXiv:2303.18223*, 2023.

[2] M. M. Solomon, “Algorithms for the vehicle routing and scheduling problem with time window constraints,” *Oper. Res.*, 35(2):254–265, 1987.
[3] P. Vansteenwegen et al., “The orienteering problem: A survey,” *EJOR*, 209(1):1–10, 2011.
[4] M. Pontiki et al., “SemEval-2014 Task 4: Aspect based sentiment analysis,” *Proc. SemEval*, 2014.
[5] P. He et al., “DeBERTaV3: Improving DeBERTa using ELECTRA-style pre-training,” *arXiv:2111.09543*, 2021.
[6] P. Lewis et al., “Retrieval-augmented generation for knowledge-intensive NLP tasks,” *NeurIPS*, 2020.